FAKULTA INFORMAČNÍCH TECHNOLOGIÍ
VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ



Project documentation

Implementation of imperative language compiler IFJ21

TEAM 069 - VARIANT I

LIST OF AUTHORS:
SUVORKIN ARTUR (XSUVOR00) 25%
FUIS LUKÁŠ (XFUISL00) 25%
NOVÁK MATĚJ (XNOVAK2Q) 25%
ČIČMIŠ DUŠAN (XCICMI00) 25%

1 Lexical analysis

A lexical analyzer is implemented as a finite automaton (see figure 1) that retrieves characters from standard input. Our version uses helper functions in the if-else tree to recognize what character it is and then calls the appropriate helper function to store additional characters in the dynamic character string, after which a token of the correct type is created and stored at the end of the dynamic token string. A token is a data structure containing the type of token and its value.

2 Syntactic analysis

Another important part of the compiler is the syntactic and semantic parser. This part of the program is located in the file syntax.c. The parser recursively traverses the entire program and checks the semantic and syntactic correctness of the program based on the LL-grammar. The parser accepts a dynamic array as input. It accesses the elements of this array using a special global variable that contains the current index of the input array. The parser then transfers the tokens to a new generated array of the same **dynamicArray** type. The work of the parser starts by running the ***parser_start()*** function, which initializes all necessary variables. After that, the remaining functions are called apart to look for errors in the program. These functions are mainly defined by individual syntactic constructs. The parser function does not return anything. If the check is successful, the next part of the program will be called from the parser.

3 Semantic analysis

The precedence syntactic analysis (PSA) is performed in the precedence.c file and its interface is given in the precedence.h file. The precedence table is used to process expressions. The PSA consists of several functions, namely precedence_analysis (the main function called by the parser), shift_operation, shift_stop_operation, reduce_operation, get_next_token (a function to get the next token to process) and token_type_to_symbol. When syntax.c encounters an expression, it calls the precedence_analysis function, which processes the entire expression using a bottom-up pass. It initializes all necessary elements and calls the first token using the get_next_token function. After that, the whole analysis starts. It iterates over the precedence table and the character retrieved from it. There are 5 characters: $<$ (stop_shift_operation), $>$ (reduce_operation), = (shift_operation), N (syntax error), E (marks the end of the PSA). If the character is $<$, we shift the whole stack to the topmost term by one and put a stop on the stack after it (for us, it's the START type defined in enum). If $>$, we check to see if there is a rule to reduce the items. If yes, the reduction is performed and if no, the analysis is terminated with error code 2. If the = character is encountered, then the item is inserted on the stack. This procedure is repeated until the $ character is encountered both on the input and on the stack of already processed items.

4 Code generation

The code generator does not implement all of the necessary functions due to lack of time and skill, however it can handle functions associated with relational and arithmetic operations, string concatenation, conditional statements (if) and so on. Most of the instructions take tokens as arguments. Instruction listing is handled by means of a dynamically expanding array, into which instructions are gradually stored during the translation of the source program. The contents of this field are printed only if the translation is successful.

5 Teamwork

We started working on the project as a whole at the beginning of October. Tasks were divided based on the preferences and programming skills of individual team members. The more complex tasks were solved as a group and vice versa, the simpler ones were solved by each member separately in their free time.

5.1 Communication

Discord served as our main and only communication platform, through which we had weekly meetings and later, when the project was approaching its final stage, we had meetings several times a week.

5.2 Version control system

We used the Git versioning system for file management and Github as a repository.

5.3 Division of tasks

- Suvorkin Artur: syntactic analysis, testing
- Novák Matěj: lexical analysis, semantic analysis, testing
- Fuis Lukáš: lexical analysis, semantic analysis, testing, diagrams
- Dušan Čičmiš: semantic analysis, code generation, testing, documentation, diagrams

6 Tables and diagrams

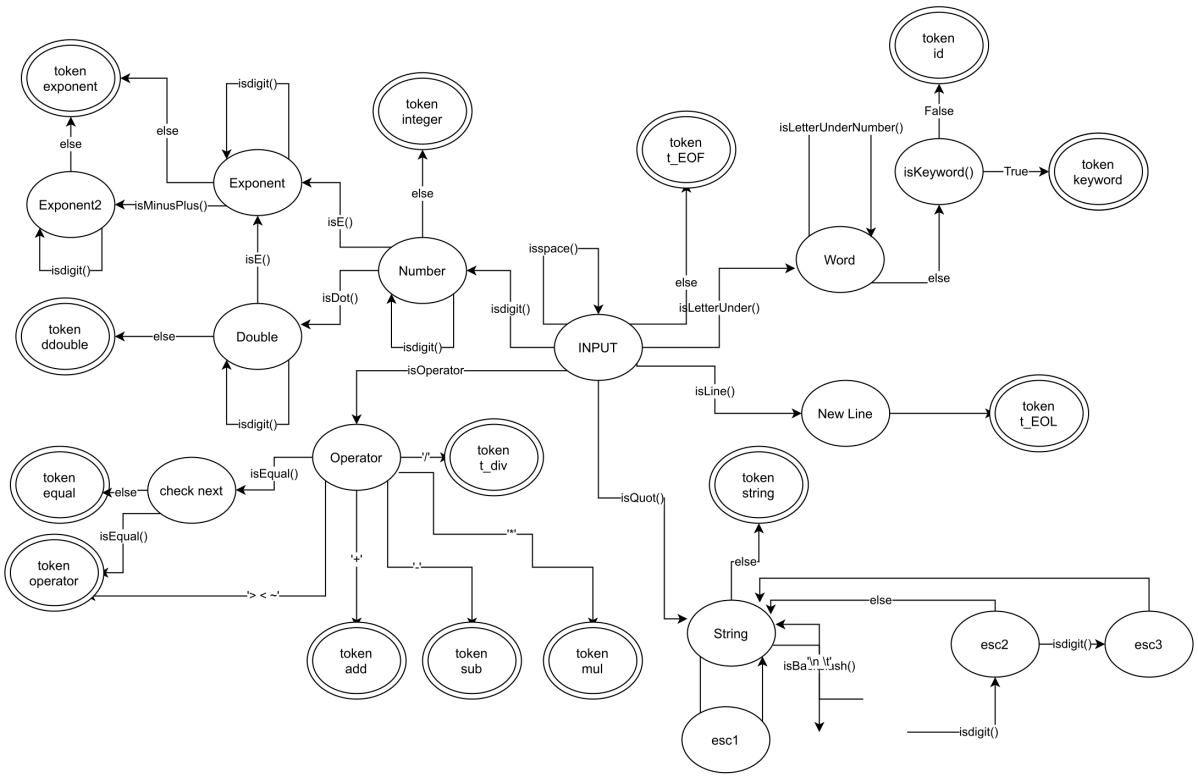


Figure 1: Deterministic finite automaton

+-	*/	(	)	log	i	$	
>	<	<	>	>	<	>	+-
>	>	<	>	>	<	>	*/
<	<	<	=	<	<	N	(
>	>	N	>	>	N	>	)
<	<	<	>	N	<	>	log
>	>	N	>	>	N	>	i
<	<	<	N	<	<	E	$

Figure 2: Precedence table